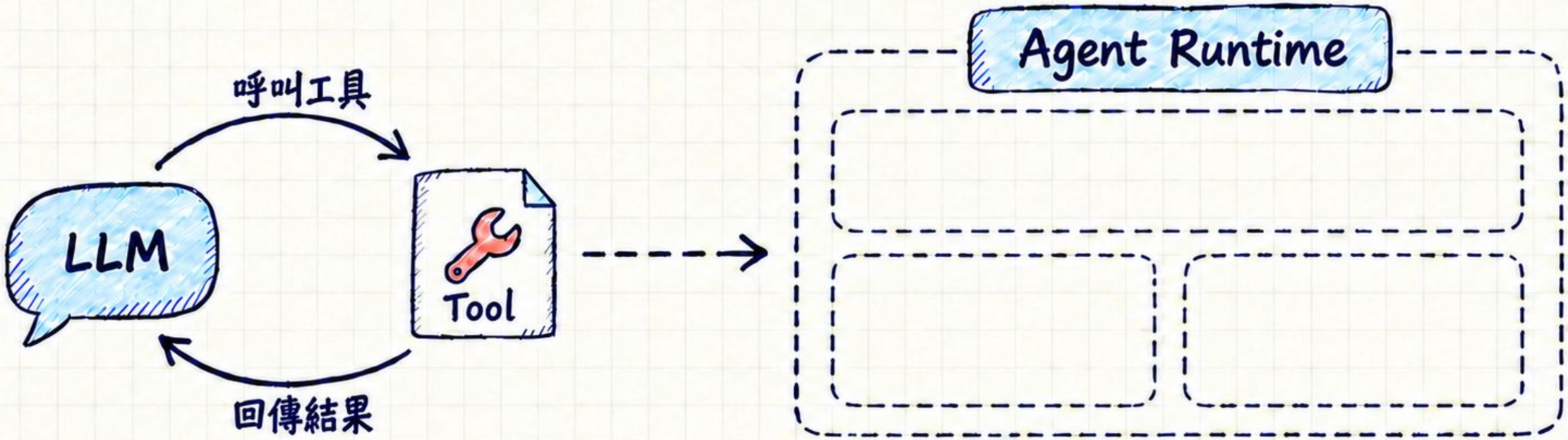
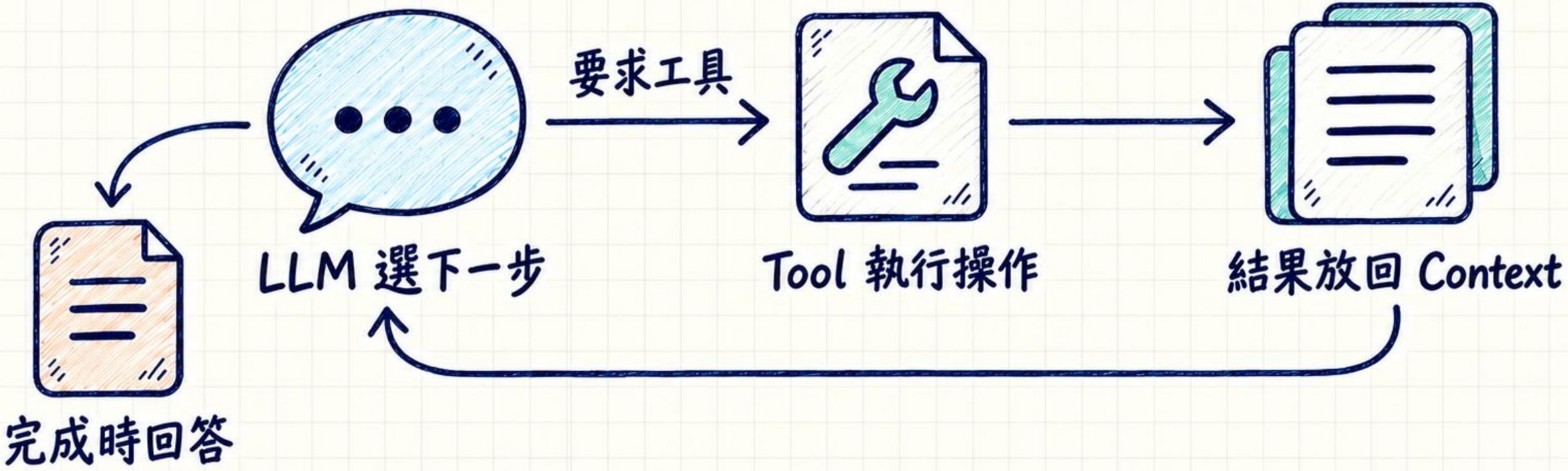
自己刻 Agent 框架，最難的不是呼叫 LLM



從 while loop，到 Agent Runtime

JamisLiao

這個 Tool Loop，幾行程式碼就能跑起來



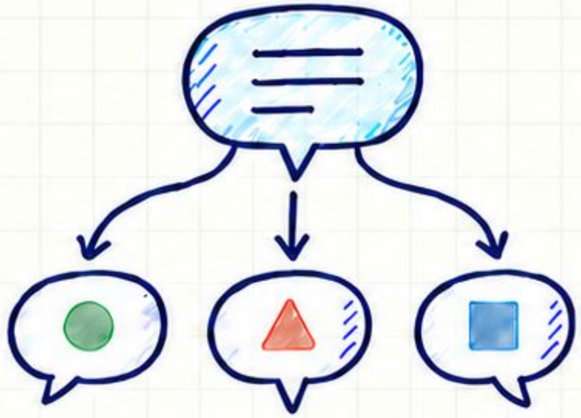
你以為這樣就結束了嗎？

真正的問題，現在才開始。

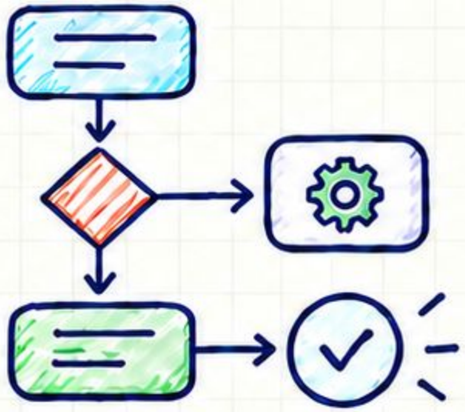
會動之後，六個問題攤開了



為什麼選擇自己刻？



模型選擇
Model Choice

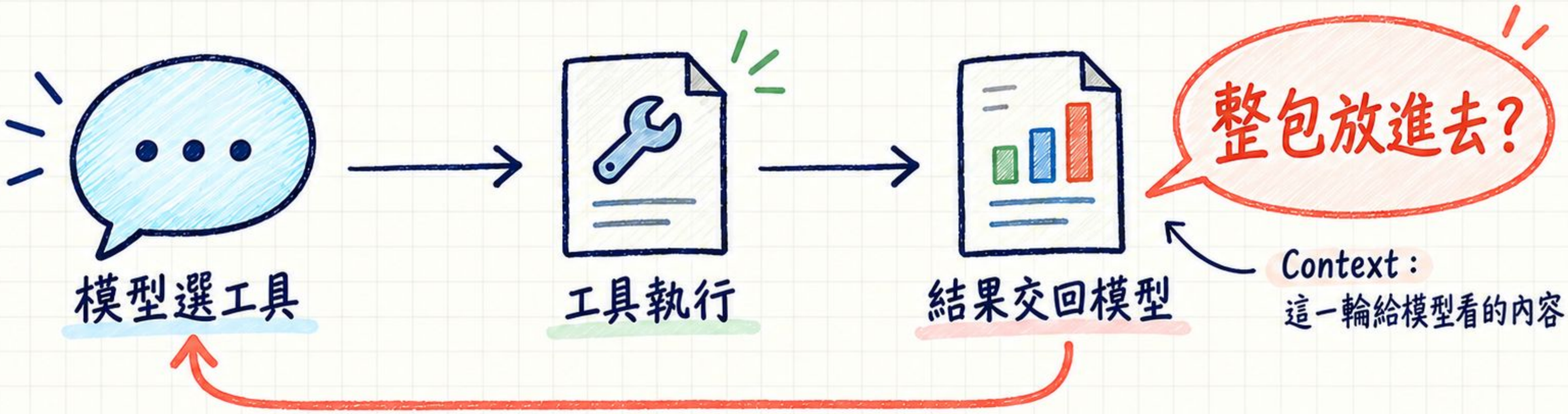


執行可控
Controllability

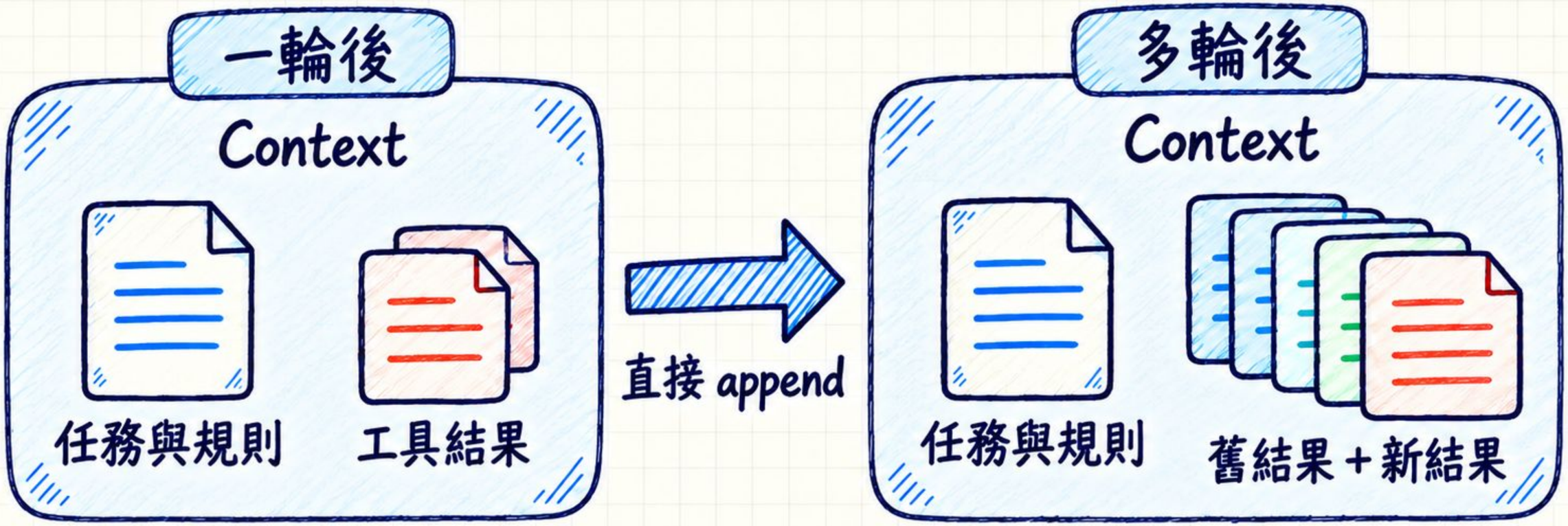


治理整合
Governance

第一個問題：工具結果，怎麼交回模型？



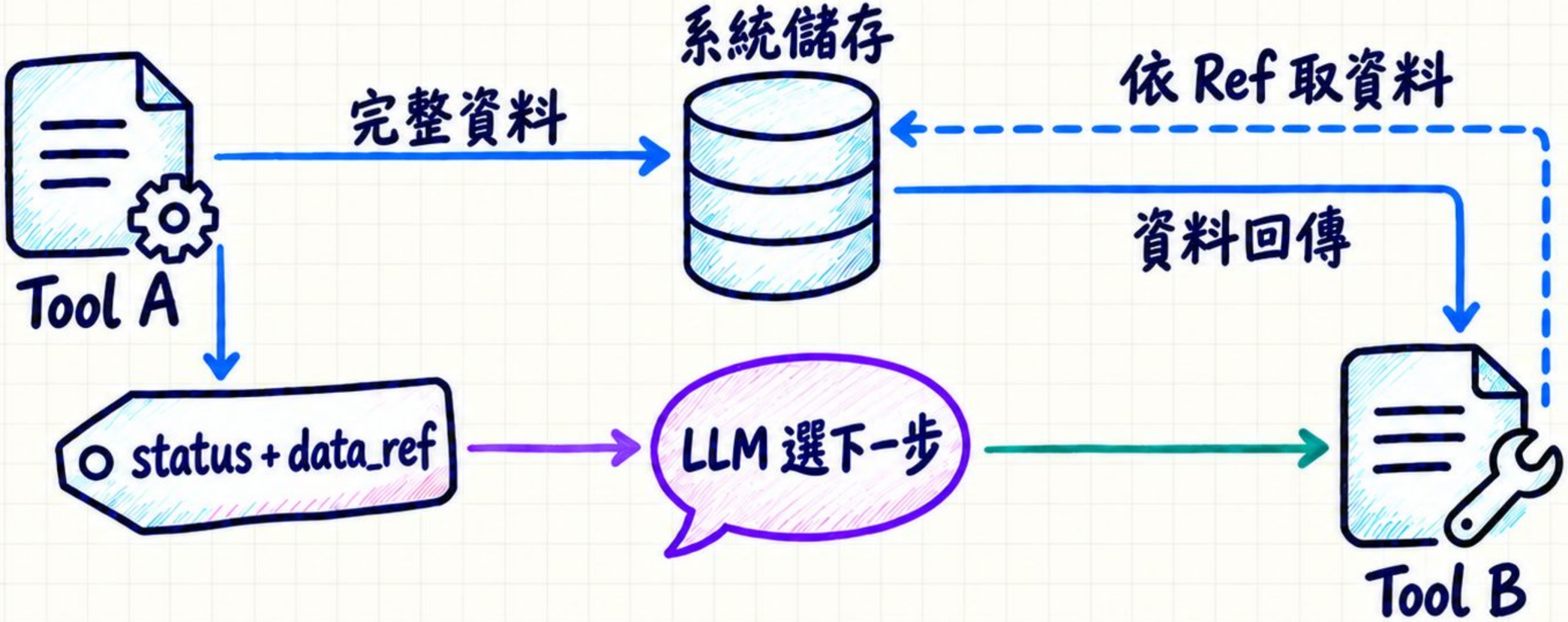
每輪都 append，Context 越堆越多



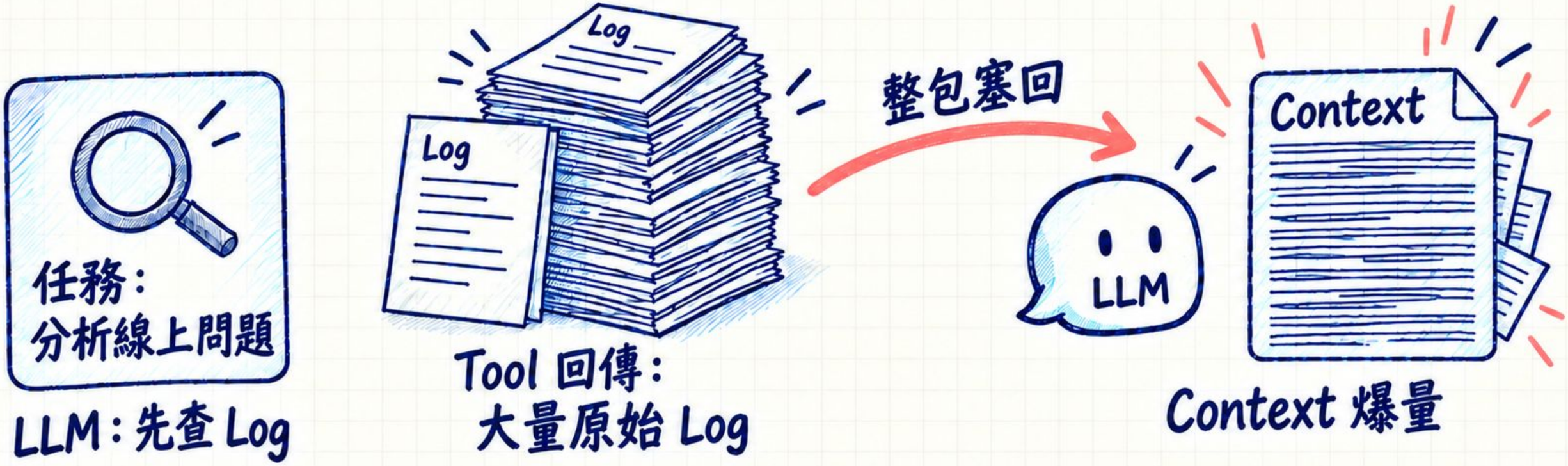
工具結果留完整，給模型的內容按需準備



模型傳 Ref，工具取完整資料



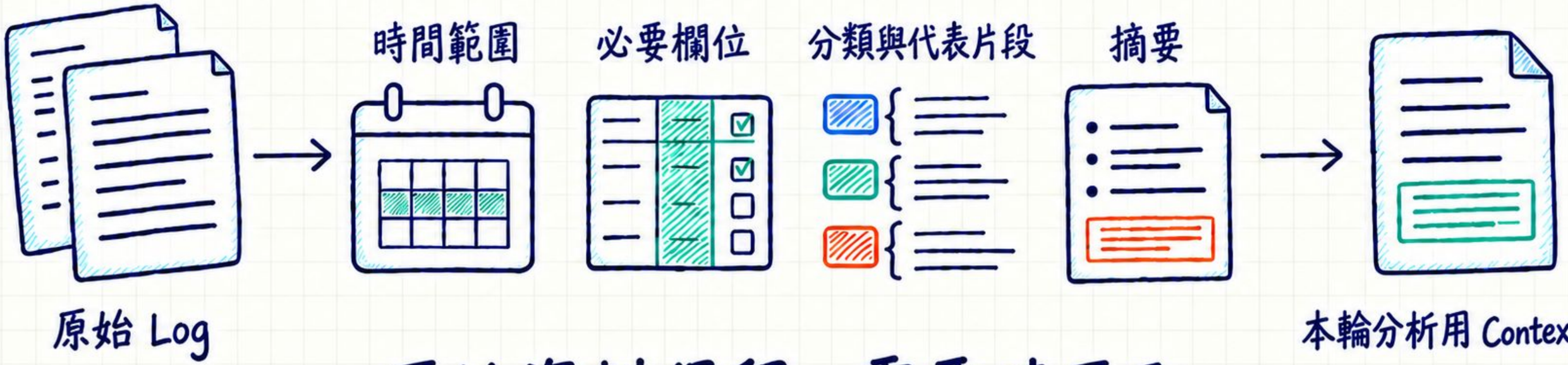
查線上問題，卻把 Context 塞爆了



查回整包，不代表要整包塞入

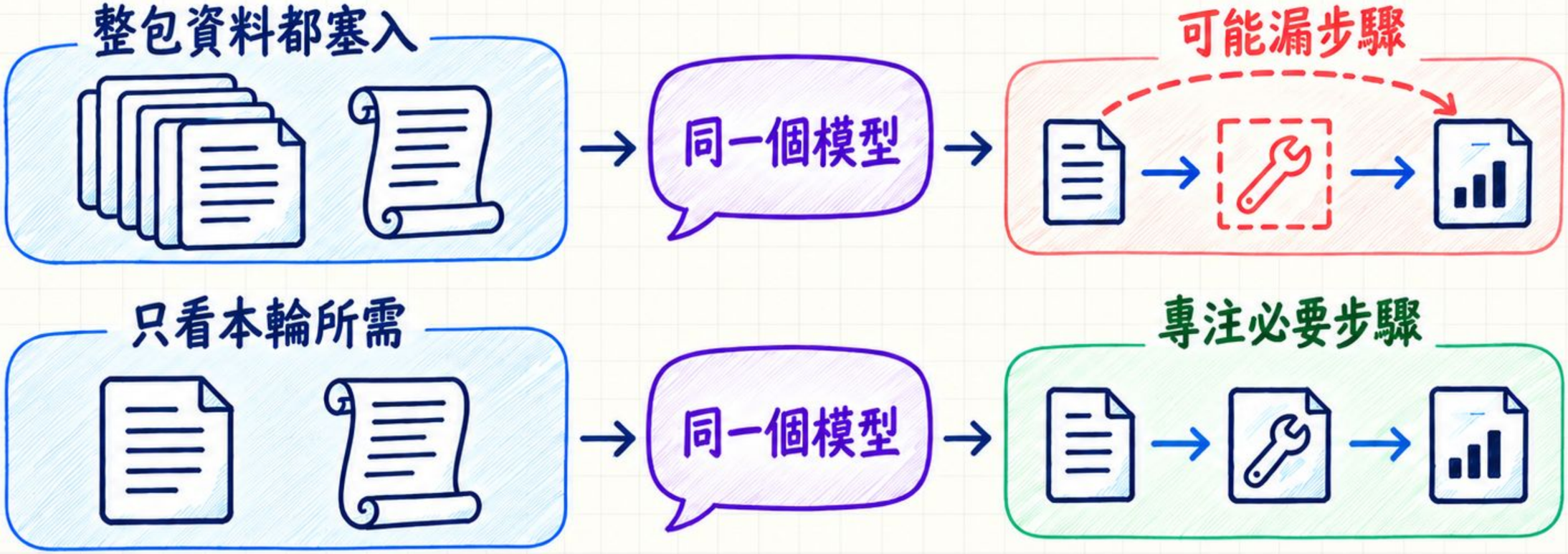
Log 先篩選，再放進 Context

依這次問題，挑出需要的 Log

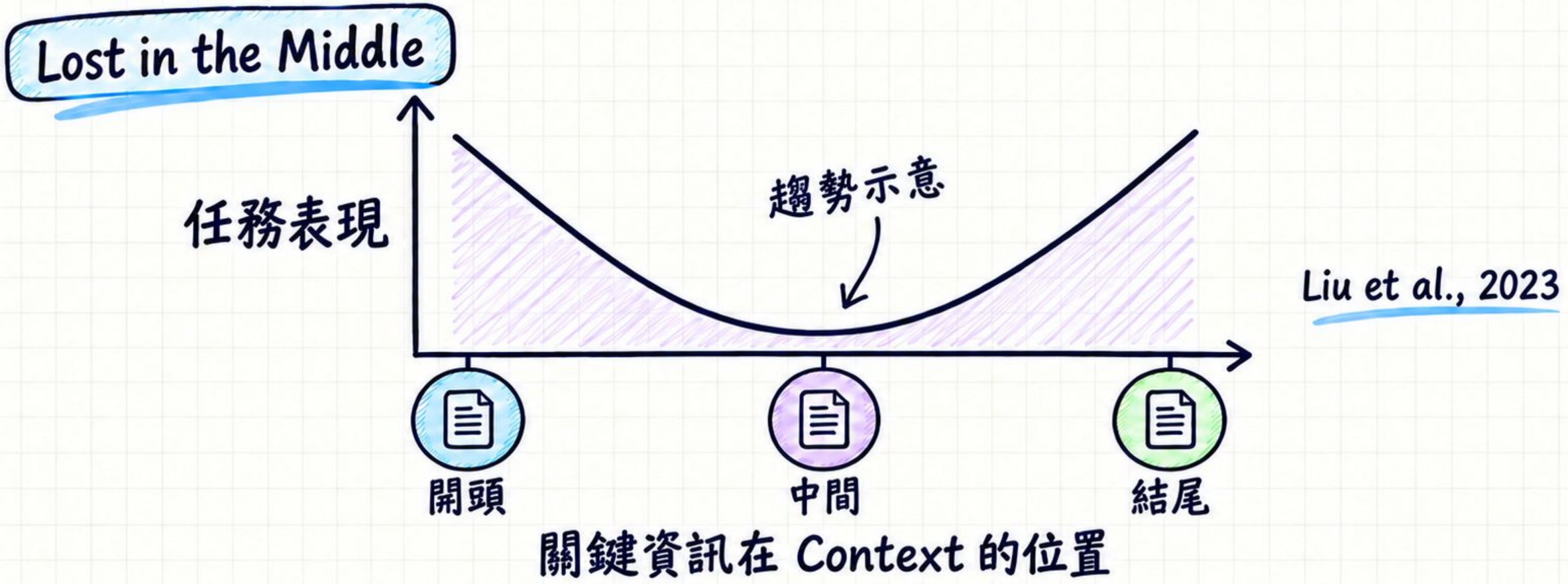


原始資料保留，需要時再取

控制 Context，不只是為了省 Token
還要讓模型顧到規則、走完流程



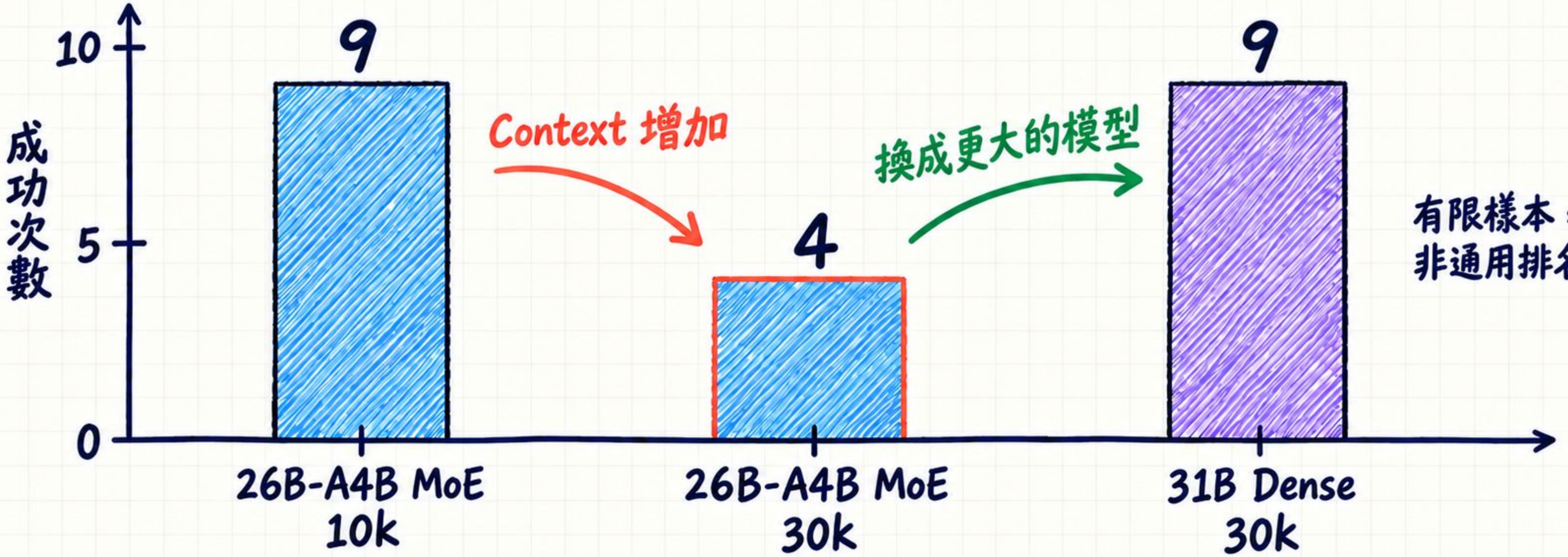
資訊放在中間，也容易被 LLM 忽略



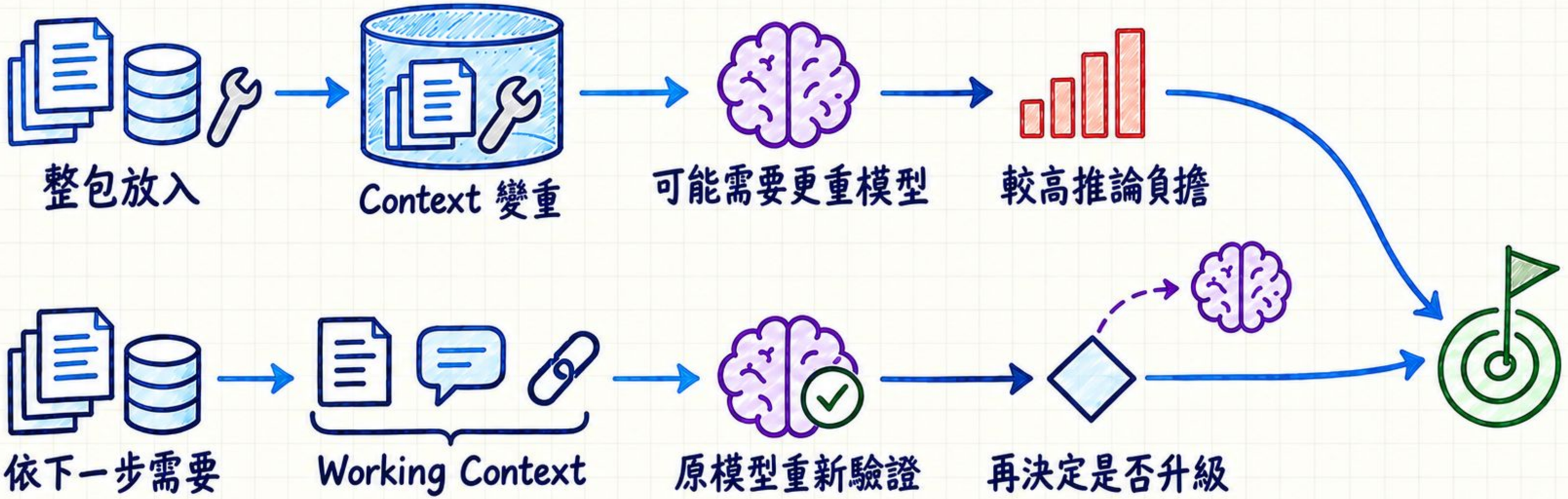
Liu et al., 2023

30k Context，迫使我換成更大的模型

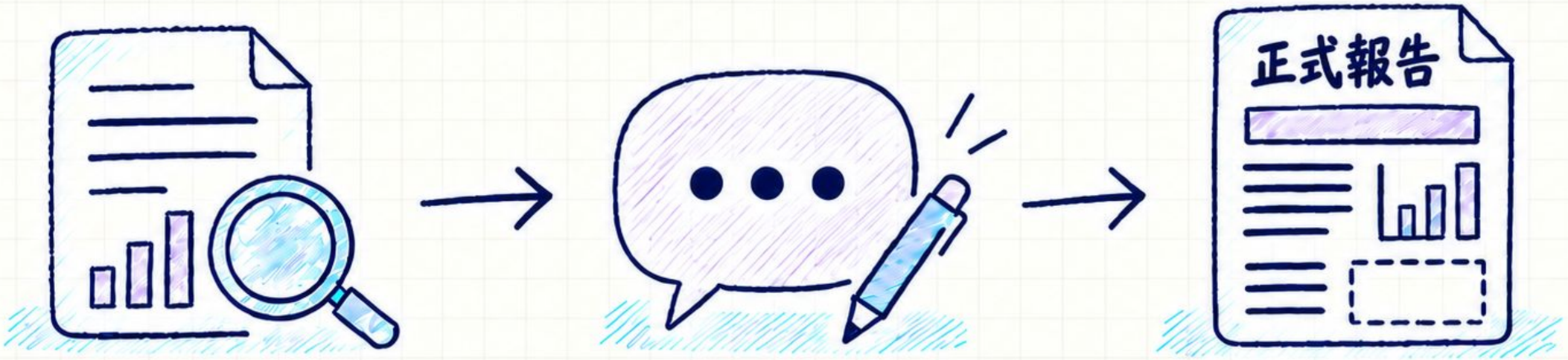
同一 Skill、測資、Tool、temperature | 每組 10 次



Context 管理會影響需要多大的模型



分析完 Log，還要交付正式報告



分析結果 → LLM 自由撰寫 → 產生報告

- 格式可能不一致
- 分類可能遺漏

正式報告，需要由 Builder 產生



分析差異　　報告 Builder　　正式報告

固定格式與必要內容，由 Builder 收斂

我已在 Skill 寫明：分析後呼叫 Builder

Skill



1. 分析完成後，呼叫報告 Builder



2. 取得 Builder 結果後，再交付報告



3. 不要自行組成最後摘要

模型卻跳過 Builder，自己寫了摘要

預期



分析差異



報告 Builder

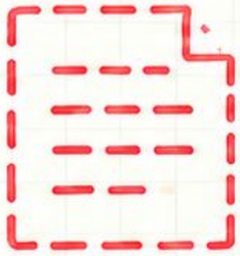


正式報告

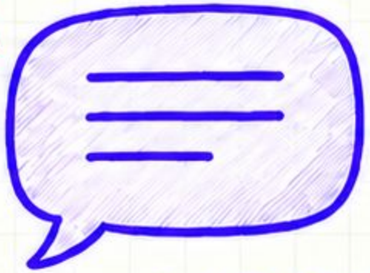
實際



分析差異



Builder 未呼叫



LLM 自行摘要

格式不符、分類漏掉

有回答，不代表流程正確完成。

Skill 已寫明

分析後呼叫 Builder

≠

實際有照做嗎？



怎麼知道 Agent 真的完成任務？

任務驗收卡



必要操作

確實呼叫 Builder



交付成果

交出 Builder 產生的正式報告

先說清楚，怎樣才算完成。

Evaluation

實際測試 LLM 的表現



準備任務與成功標準　讓 LLM 實際執行　對照結果是否達標

不是猜它會不會，而是實際測它做得如何

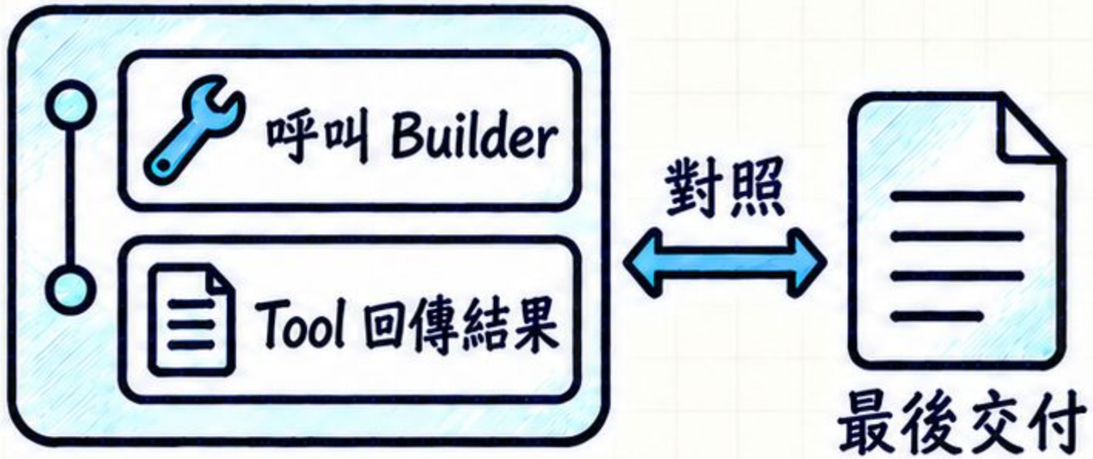
把實際執行，對回任務要求

交付成果



指定報告有交付嗎？

工具呼叫紀錄



Builder 有呼叫嗎？
交付的是它的結果嗎？

Evaluation 要看結果，也要看執行過程

Outcome Evaluation

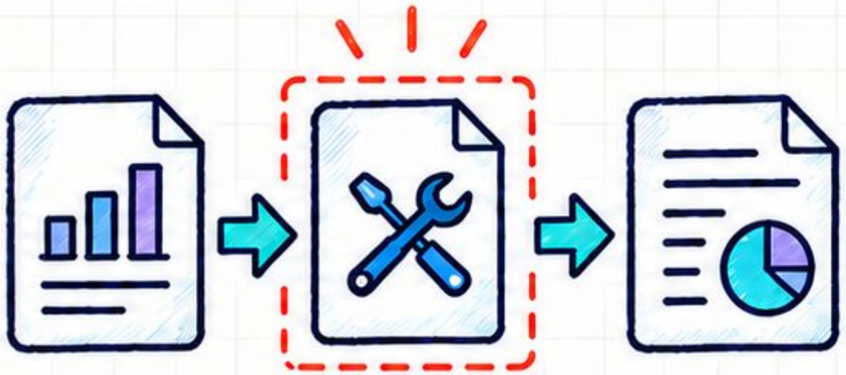
結果評估



報告格式符合要求？
分類內容完整？

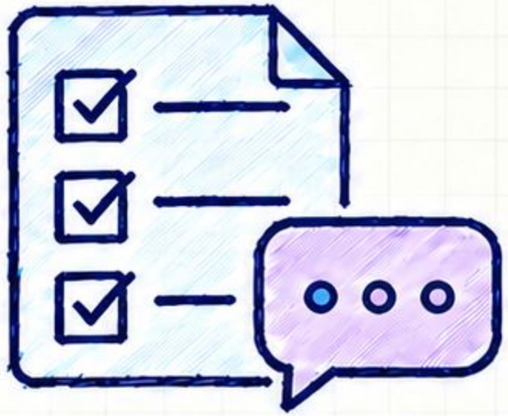
Trajectory Evaluation

執行軌跡評估



Builder 有呼叫？
必要步驟有漏掉？

這些要求，要怎麼評估？



LLM-as-a-Judge
模型評審

依評分準則判斷
內容與軌跡的合理性



Code-based Evaluation
程式檢查
Trajectory Matching

核對工具、參數與格式
比對預期執行軌跡

把明確要求，做成程式檢查



固定條件重跑，觀察流程是否穩定

固定 Skill

固定測資

固定 Tool

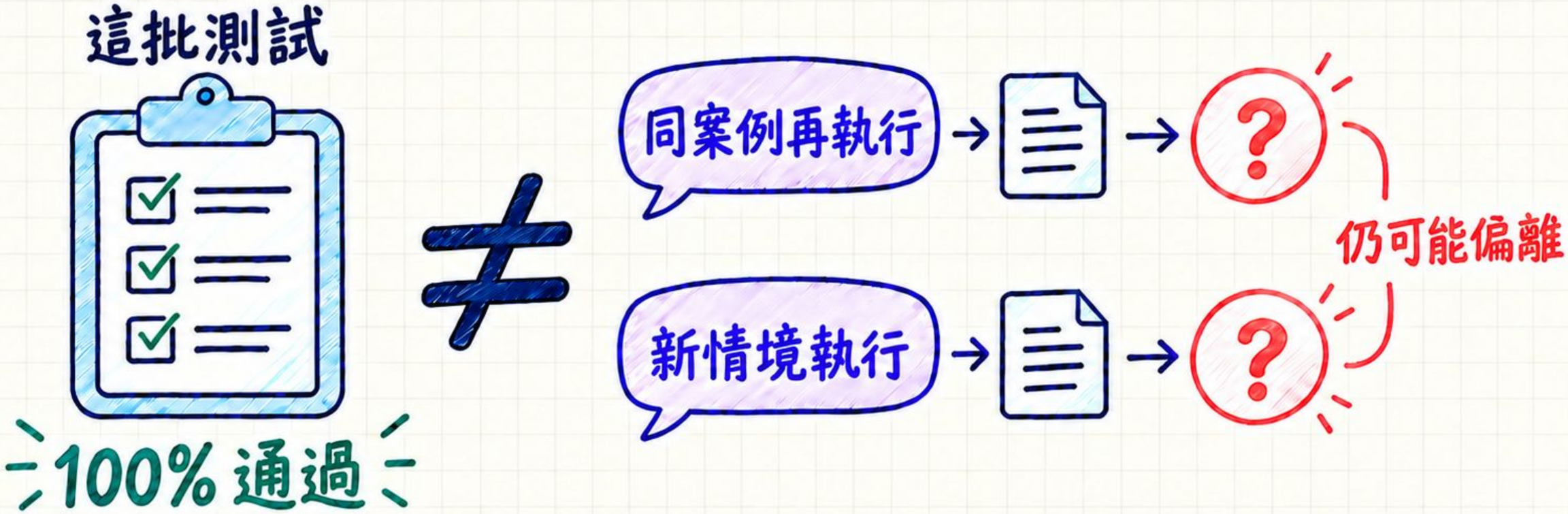
固定模型設定

同一任務

重複執行

比對必要操作

測試全過，不代表下次一定照做



不能接受的錯，就不能只靠模型遵守

可接受偶發錯誤



Skill + Evaluation

一次都不能接受



系統檢查 + Guardrail

把不可接受的錯，變成可檢查的條件



日期不能算錯 →  採用 Tool 計算結果

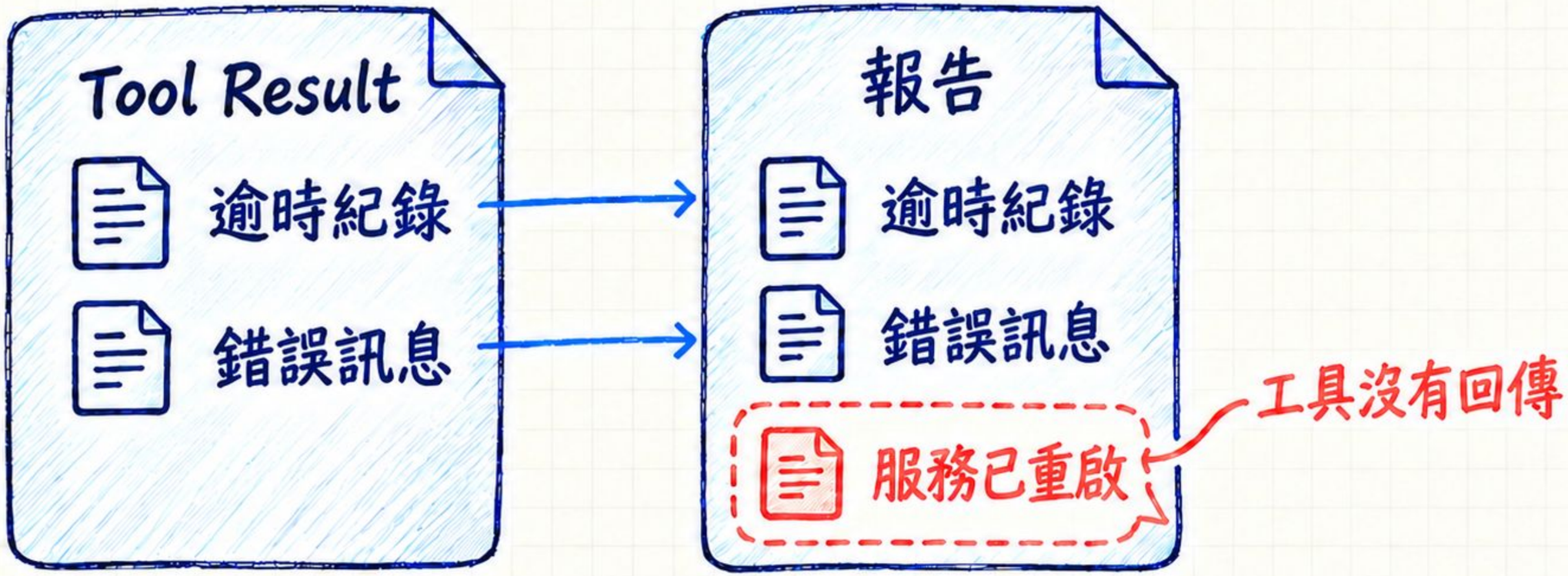


輸出結論要有證據 →  輸出前檢查支持依據

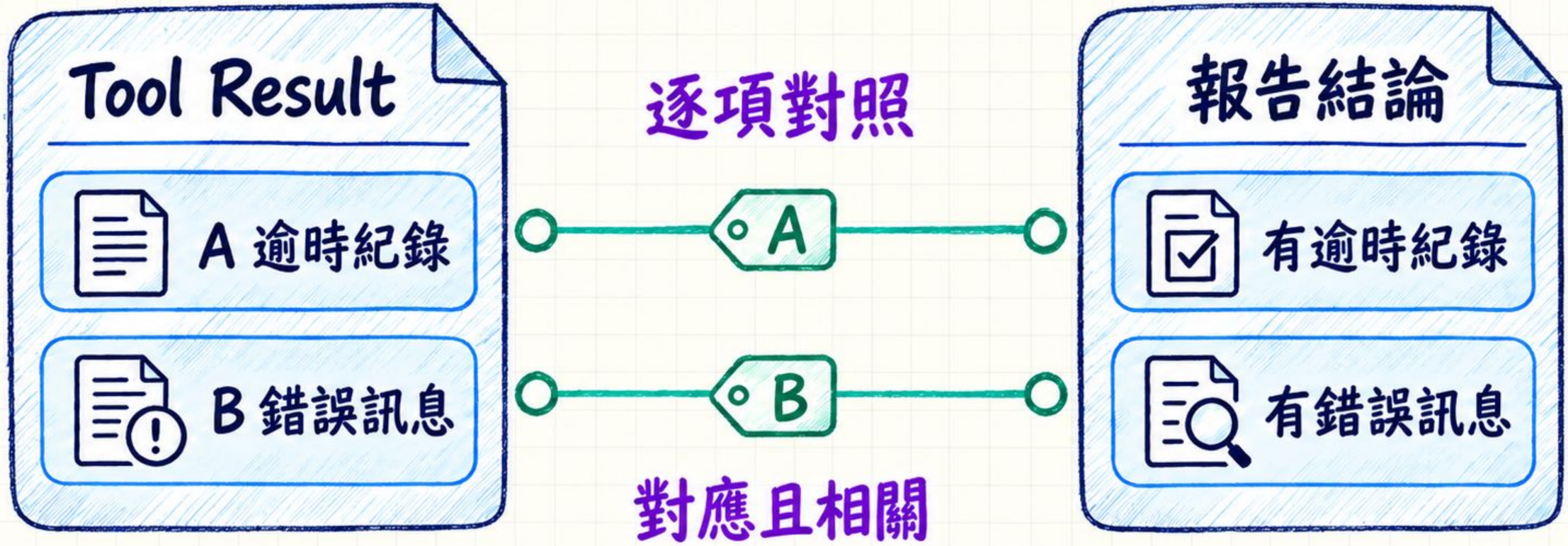


權限不足不能修改 →  修改前由程式檢查

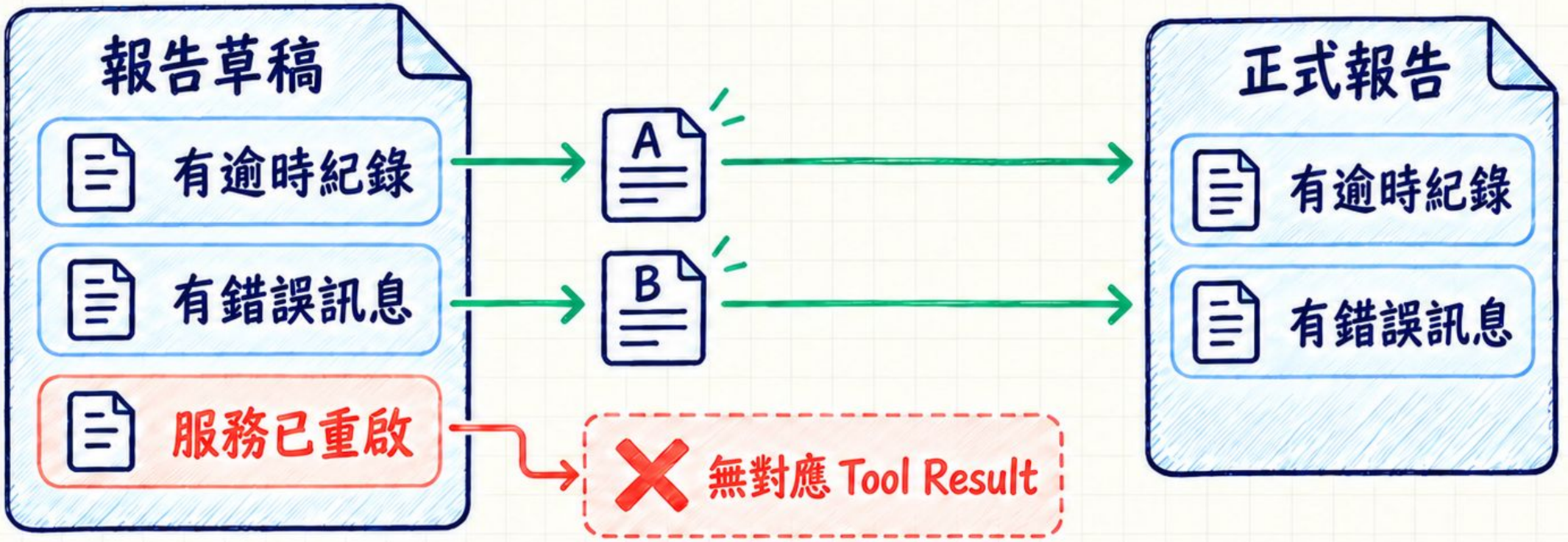
同一份 Log 報告，
還可能多出沒有依據的內容



每一項結論，都要對回 Tool Result



沒有對應證據的結論，不能進正式報告



Agent 要做什麼，有時需要人先確認

Human-in-the-Loop（HITL）



⚠ 確認理解 ≠ 批准操作

先確認目標與範圍，再開始任務

Agent 理解的任務

- 目標：A 類
- 排除：B 類
- 範圍：最近三天
- 狀態：失敗項目

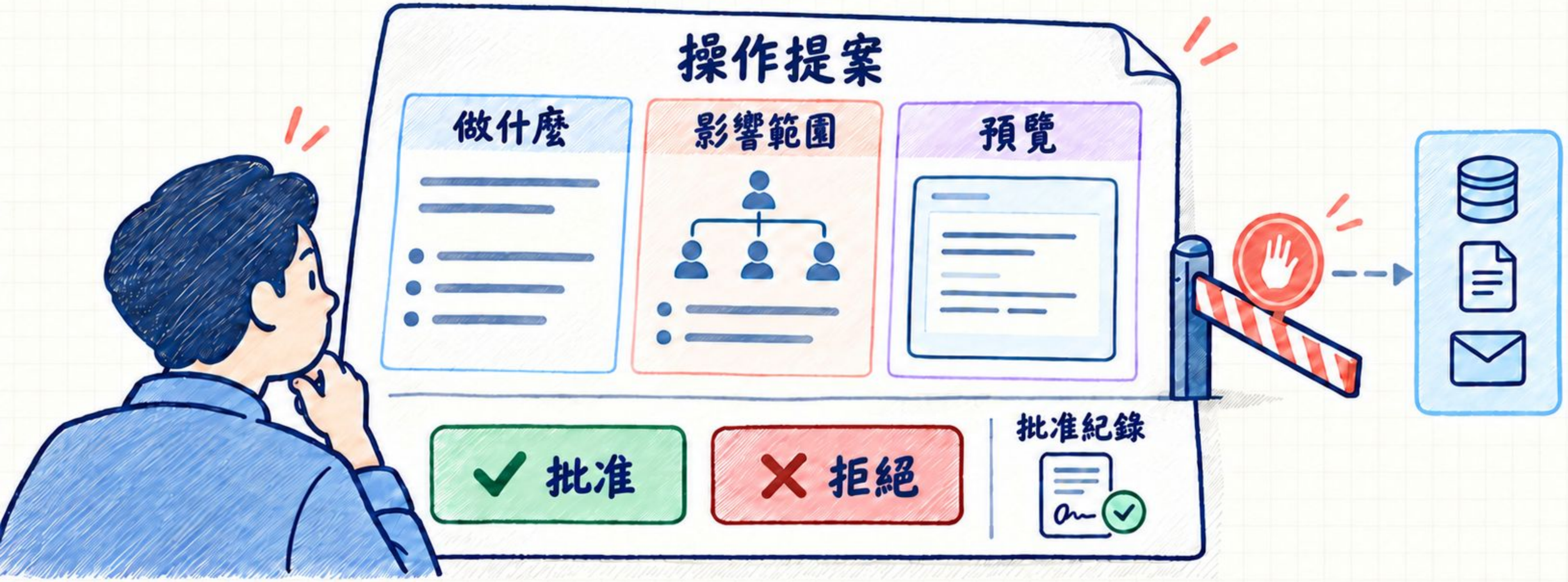
確認／修改／退回

再開始呼叫工具



工具成功 ≠
任務正確

寫入、扣款或通知前，先取得批准



批准後，執行前仍要重新檢查條件



除了防錯，也要能追查出錯原因

看見問題



原因還待確認



看見錯誤 ≠ 知道原因

再跑一次，不能代替回看出錯那次

出錯那次



重新執行



先留下 AI 能協助追查的資訊



實際 Context

當輪輸入與指示



模型回應

文字與 Tool 呼叫



Tool 執行

參數、結果與錯誤



版本與設定

模型、Skill 與參數

對得上同一次任務、同一輪呼叫

漏了 Builder，要對照哪些紀錄？



當輪 Context

有送入 Builder 指示嗎？



模型回應

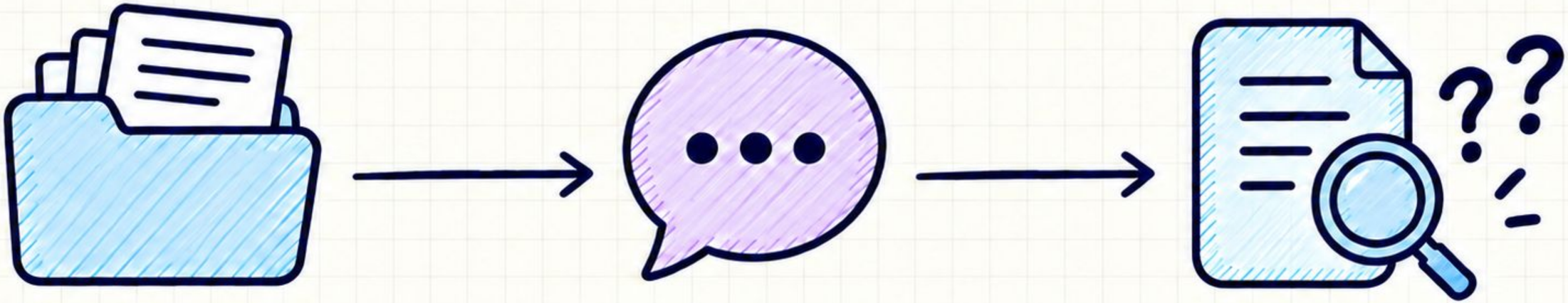
有提出 Builder 呼叫嗎？



Tool 執行紀錄

呼叫有實際執行嗎？

把執行紀錄交給 AI，協助找線索



提出修改，再用 Evaluation 驗證

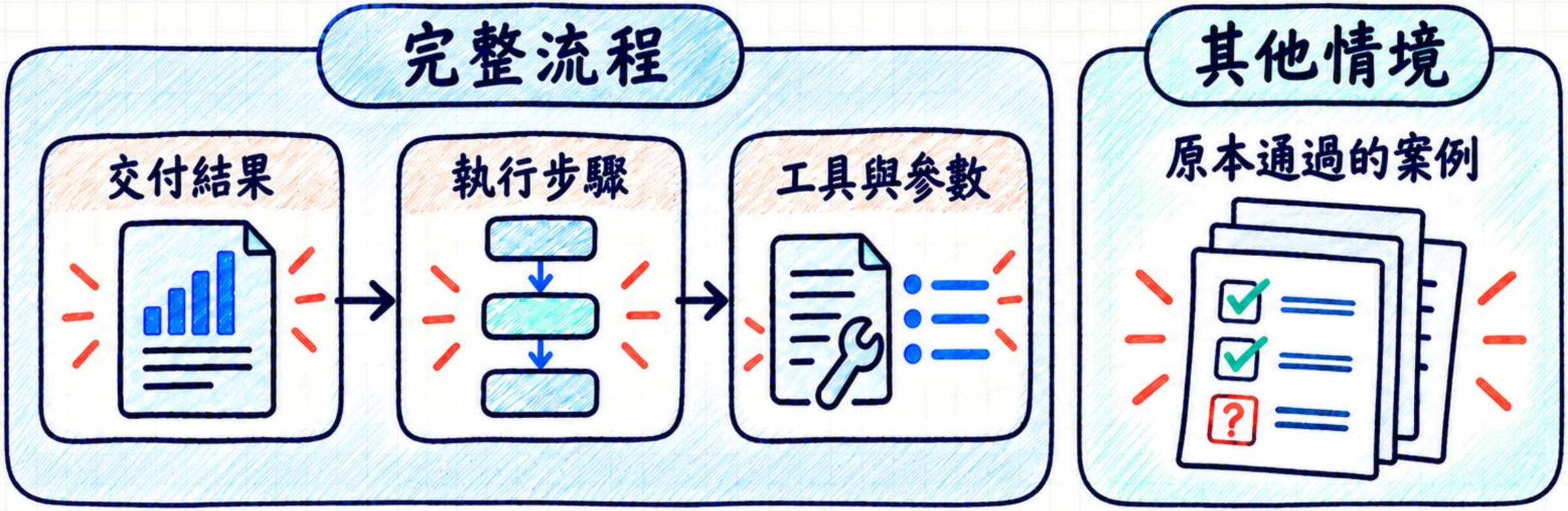


改完有沒有改善，要再測一次

修好一個問題，
也可能弄壞其他情境。



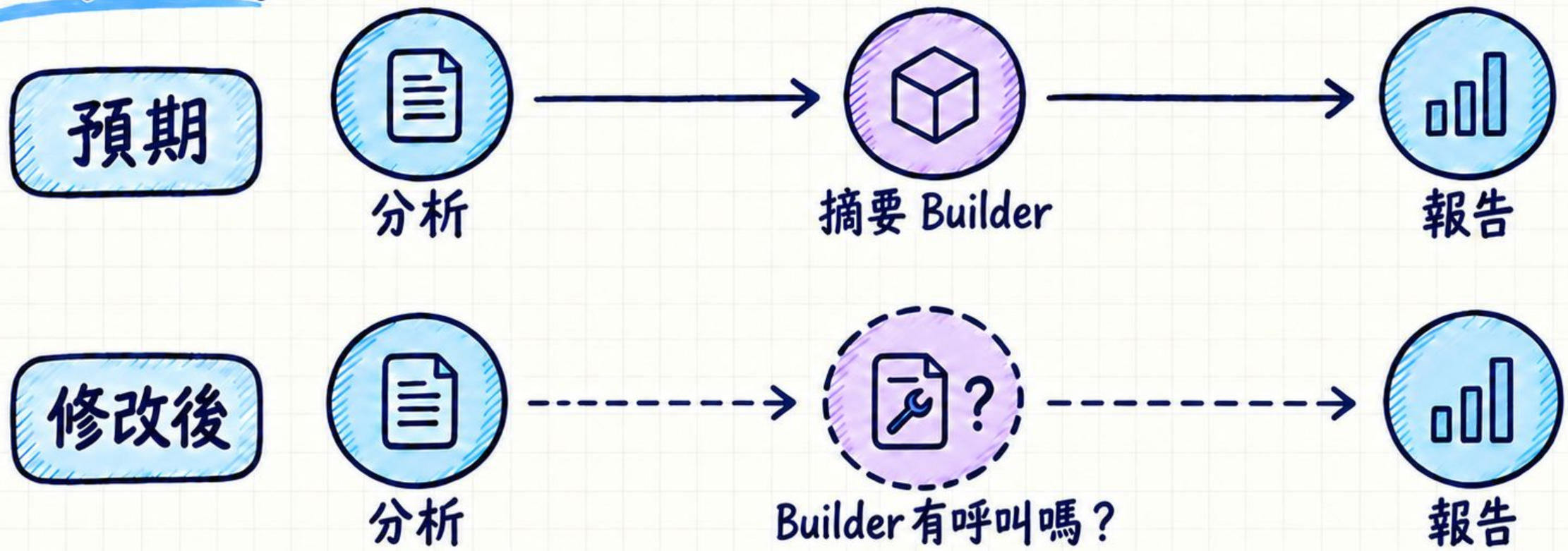
改完 Skill，完整流程與其他情境都要重測



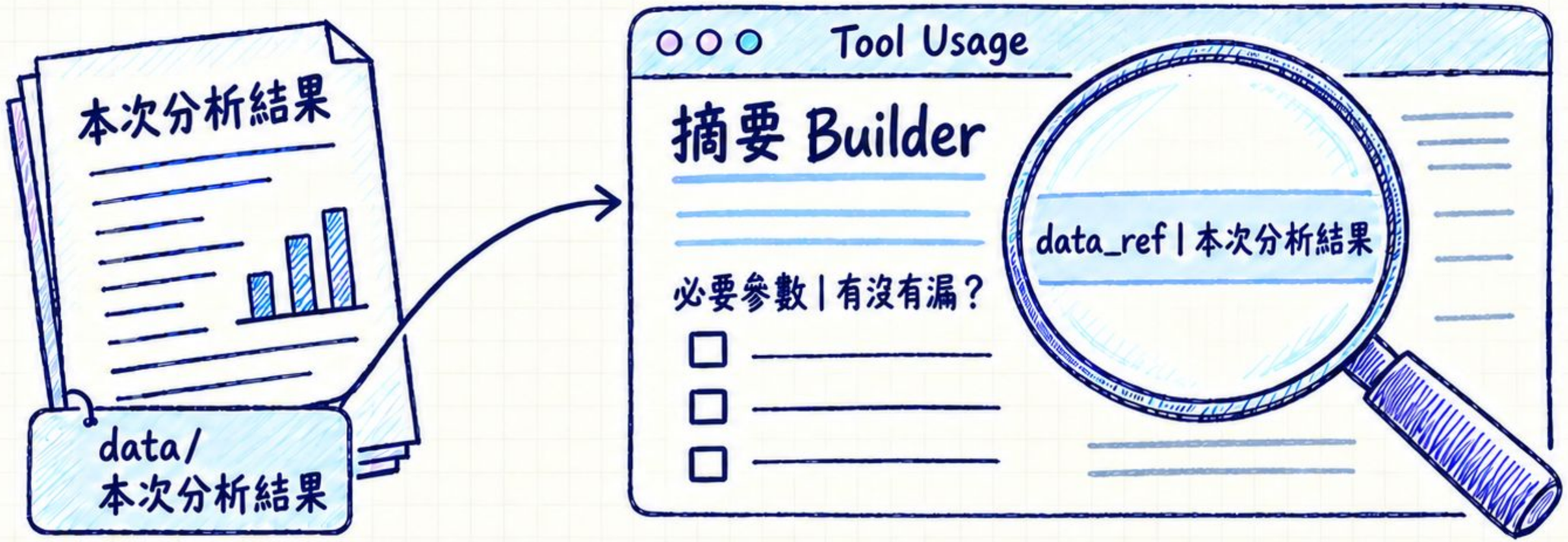
不只看原本的問題，也要確認其他情境沒有退步

漏掉的 Builder，這次有呼叫嗎？

Trajectory

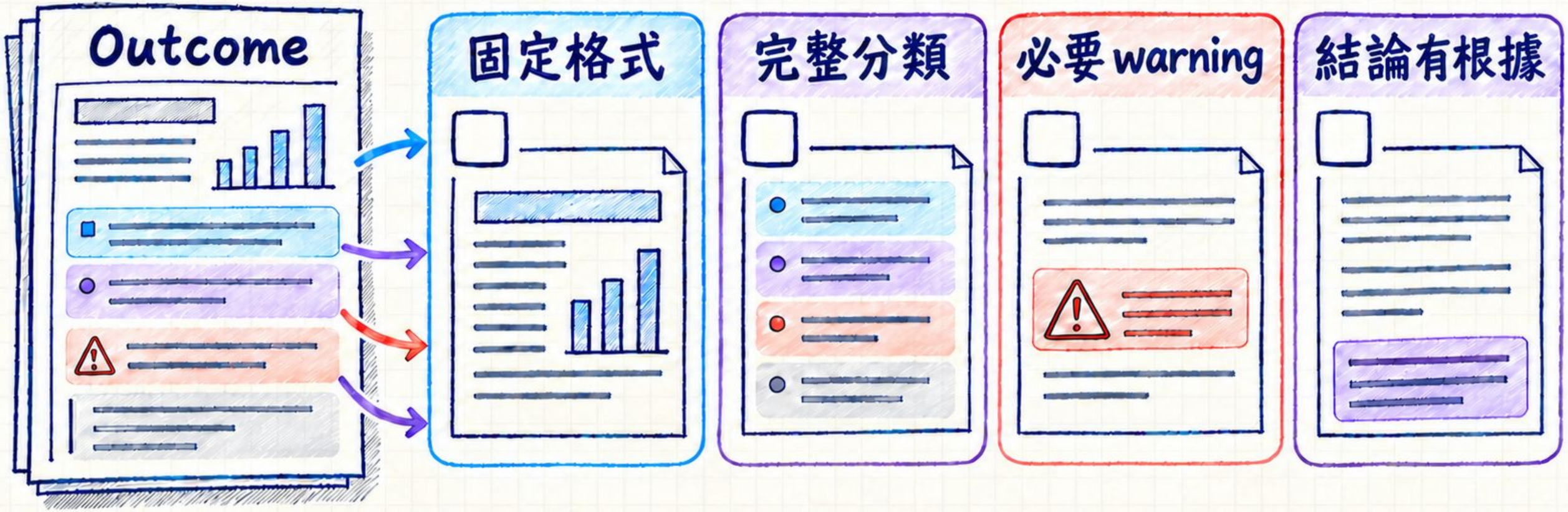


呼叫了 Builder，傳入的資料也對嗎？



呼叫正確 ≠ 使用正確

輸出報告，格式與內容也要符合要求



驗收修改：哪些用程式，哪些用 Judge？

明確規則 → 程式

工具呼叫

摘要 Builder

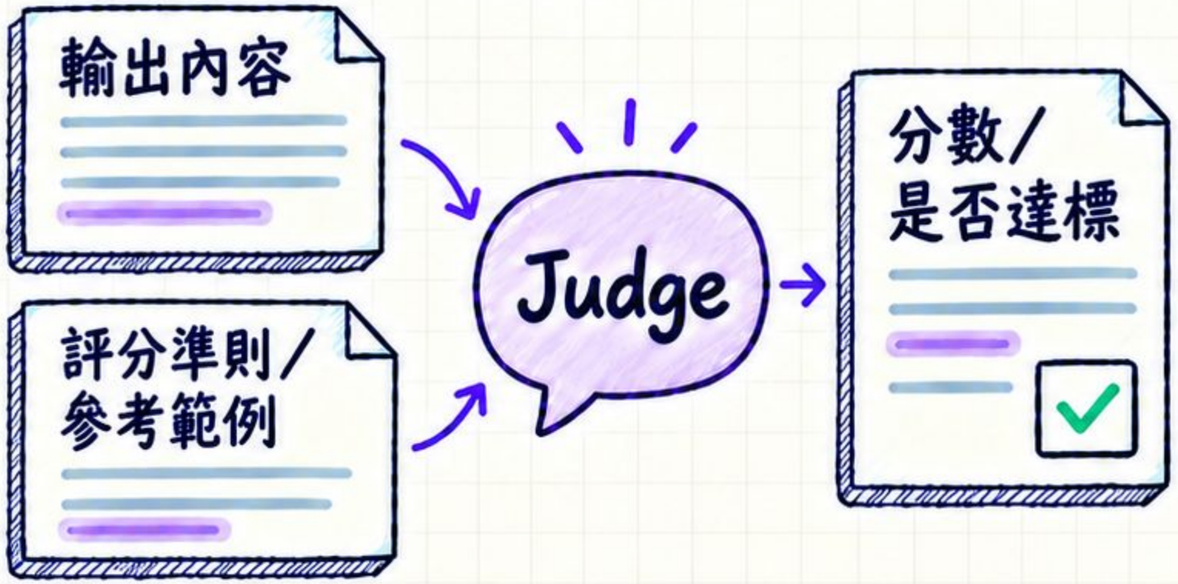
data_ref

參數檢查清單

- [ ] Builder 有呼叫嗎？
- [ ] Ref 有效嗎？
- [ ] 必要欄位齊全嗎？

語意判斷 → Judge

Judge 也可能判錯

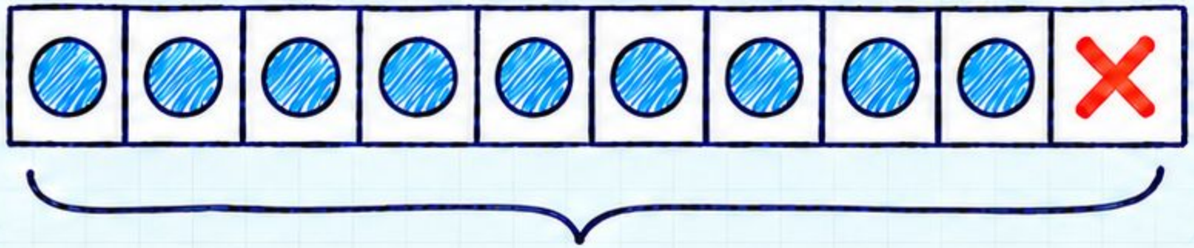


- [ ] 是否符合需求？
- [ ] 是否涵蓋重點？

通過幾次，才接受這版修改？

門檻示例，非新實測

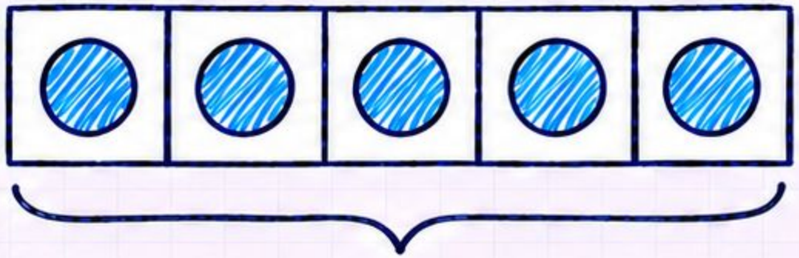
依風險與成本設定



10 次成功 9 次

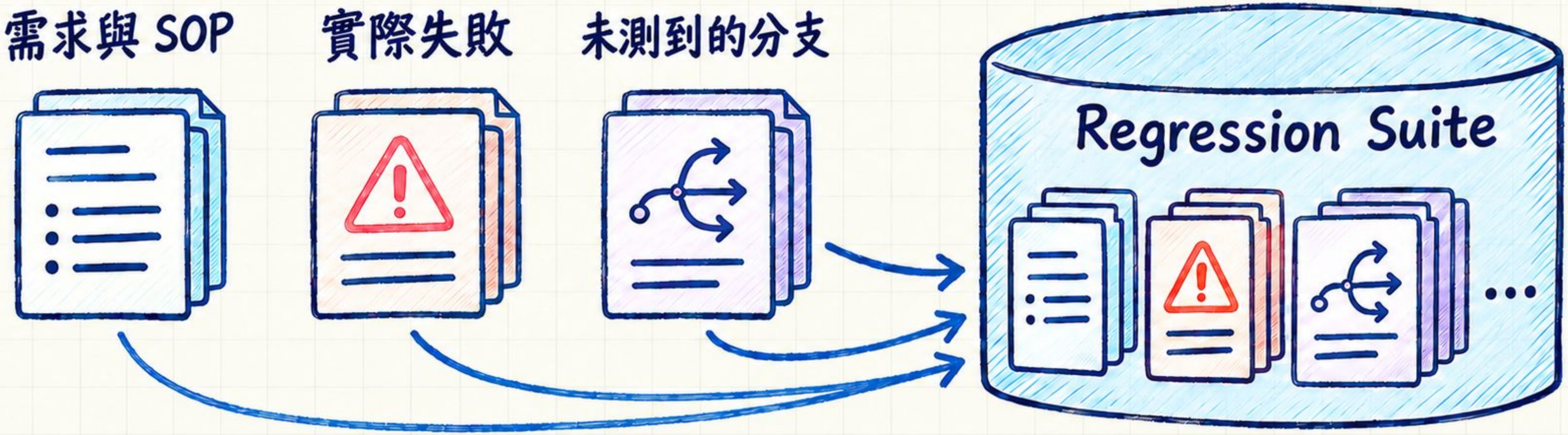
門檻示例，非新實測

依風險與成本設定



或 5 次全部成功

踩過的坑，留進回歸測試



每次修改，重測既有能力

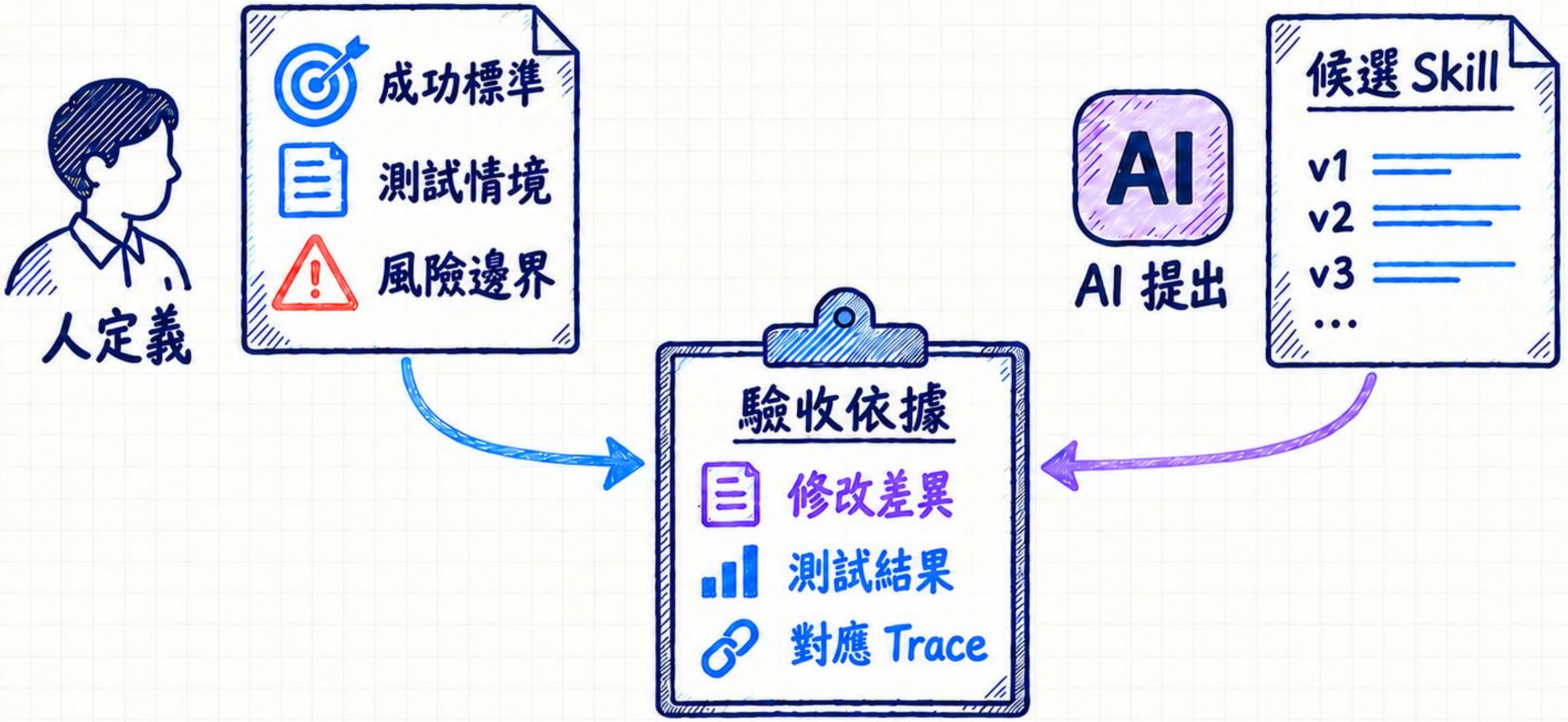
從第一版 Skill 到功能完成，我沒有手寫過一行 Prompt。



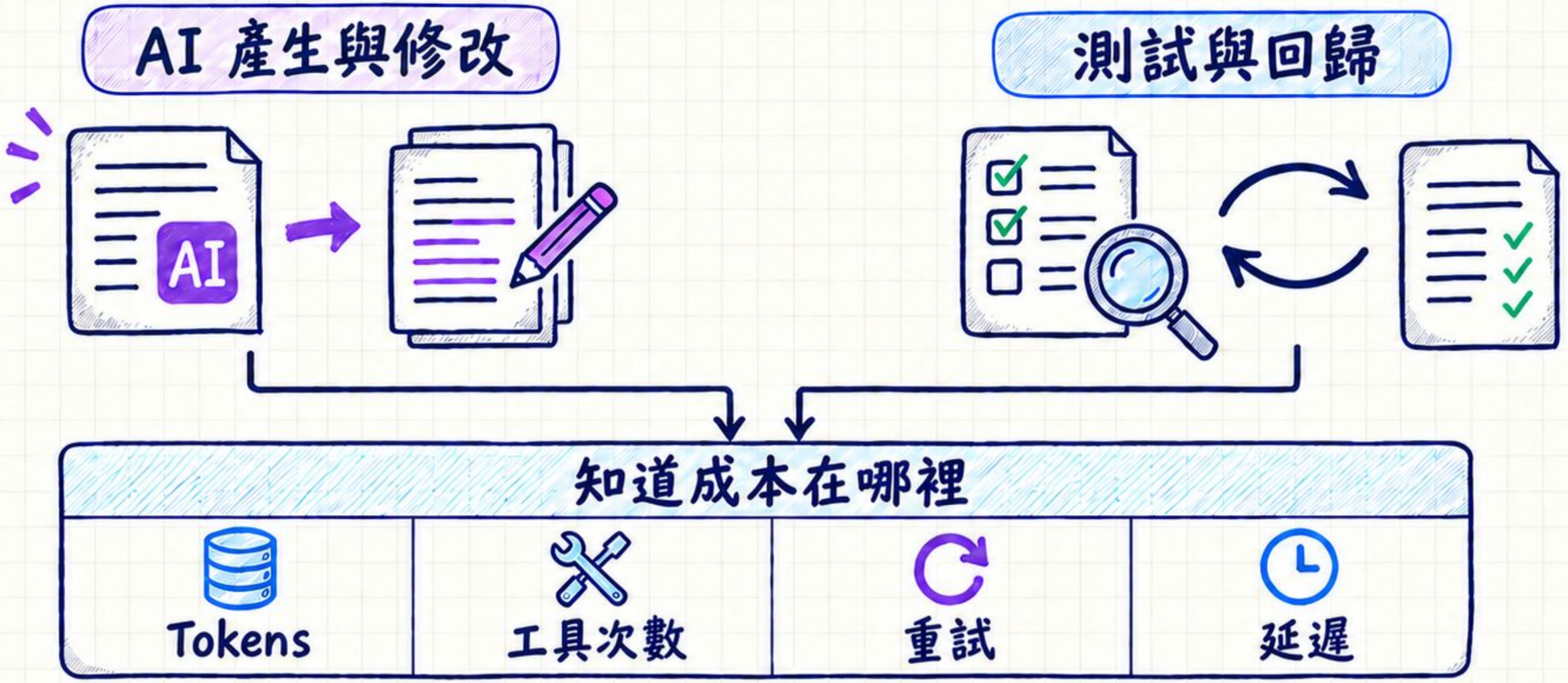
Eval 的價值不是分數，
而是讓 Skill 能被持續修正。



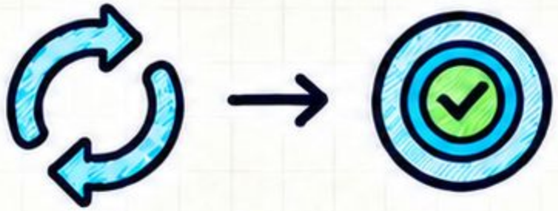
人定義成功標準，修改用測試結果驗收



每輪產生、測試與修正，都要留下成本紀錄



寫出一個 Loop 很簡單，
讓一個 Loop 能夠正確地完成任務，
才是困難的工程問題。



解決這個問題，
不能只靠最新的模型。



GPT-6
Astra



Claude
Opus 5.5



Gemini
4 Argon

模型會越來越聰明，

但是你的口袋沒有越來越深。

